



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 1

Team No.: 3

Project Title: Linux Process Debugging & Fault Diagnosis Toolkit

Team Members

Roll Number	Student Name	Signature
2520030068	G.Neha Deepika	
2520030103	Poojitha Sai	
2520030541	Poojitha Musunuri	

1. Abstract

Linux Process Debugging & Fault Diagnosis Toolkit is a Linux-based systems programming project designed to assist in identifying and analyzing common process execution failures. Debugging low-level programs can be difficult because faults such as segmentation faults, abnormal signal termination, and non-terminating processes are closely related to operating system process and memory behavior.

The proposed system will execute and monitor target programs as controlled processes and analyze their execution status, termination conditions, signals, and selected process information. It will use Linux/POSIX mechanisms such as `fork()`, `exec()`, `waitpid()`, `ptrace()`, signals, and `/proc` process information to observe and diagnose program behavior. The system will classify common execution failures and generate a structured diagnostic report containing the detected fault, process status, relevant signal information, and possible cause.

The project will also include controlled test programs representing normal execution, segmentation faults, infinite loops, and signal-based termination. These programs will be used to validate the debugging and fault-diagnosis capabilities of the system.

The expected outcome is a command-line Linux debugging toolkit that demonstrates practical application of process creation, execution, monitoring, signals, memory-error analysis, and Linux system programming concepts while providing understandable diagnostic information to the user.

2. Problem Statement

Debugging low-level Linux programs requires understanding process creation, execution, termination, signals, and memory-related failures. Conventional debugging tools can provide detailed information, but beginners may find the relationship between program failures and operating-system behavior difficult to understand. Therefore, there is a need for a Linux-based system that can execute controlled programs, observe their process behavior, identify abnormal termination or execution conditions, and present understandable diagnostic information. The proposed project addresses this problem by developing a process-oriented debugging and fault-diagnosis toolkit using Linux/POSIX system programming mechanisms.

3. Objectives

Objective 1

To execute and monitor target programs as controlled Linux processes.

Objective 2

To detect and analyze abnormal process termination, including signal-based failures such as segmentation faults.

Objective 3

To identify abnormal execution conditions such as non-terminating or runaway processes.

Objective 4

To generate structured diagnostic information explaining the detected process state, fault condition, and possible cause.

4. Proposed Methodology

The system will be implemented as a command-line Linux application in C. The user selects or provides a target program for analysis. The system creates a controlled child process using `fork( )` and executes the target using an appropriate `exec( )` function. The parent process monitors the child using `waitpid( )` and signal-related mechanisms. Where required, `ptrace( )` will be used to observe and control the target process during debugging. Process information will be obtained from Linux interfaces such as `/proc`. The system will identify normal completion, abnormal signal termination, segmentation faults, and prolonged execution. Based on the observed process behavior, it will classify the fault and generate a diagnostic report. Controlled test programs will be used to evaluate the system under different execution scenarios.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / API	Purpose
<code>fork()</code>	Creates a controlled child process for executing the target program
<code>exec()</code>	Loads and executes the target program
<code>waitpid()</code>	Allows the parent to monitor child-process termination/status
<code>ptrace()</code>	Provides process tracing and debugging control
<code>sigaction()</code>	Handles and analyzes process signals
<code>kill()</code>	Sends signals for controlled process management
<code>/proc</code>	Provides Linux process information
<code>open()</code> , <code>read()</code> , <code>close()</code>	Used for reading relevant system/process information
Process lifecycle	Used to analyze creation, execution and termination
Signals	Used to identify abnormal process termination

6. Individual Contribution

Roll No.	Student	Responsibility
252003068	G. Neha Deepika	Process execution, process lifecycle monitoring and Linux <code>/proc</code> integration
252003010	Poojitha Sai	Fault detection, signal handling and debugging/tracing functionality
252003054	M. Poojitha	Diagnostic reporting, test cases, integration and documentation

7. Tools / Platforms / Software Used

Tool / Platform	Purpose
Ubuntu Linux	Development and execution environment
C	Core systems programming implementation
GCC	Compilation
POSIX APIs	Process and signal programming
Linux <code>/proc</code>	Process information
Git	Version control

GitHub	Project repository and collaboration
VS Code / Nano	Source-code editing
Terminal	Compilation, execution and testing

8. Expected Outcome

The project is expected to produce a functional Linux command-line debugging and fault-diagnosis toolkit capable of executing and monitoring target programs, detecting abnormal process termination and selected execution failures, analyzing relevant signals and process information, and generating structured diagnostic reports. The system will demonstrate practical use of Linux/POSIX process management, signals, tracing, and system-programming mechanisms. Controlled test programs will be used to validate normal execution, segmentation faults, signal-based termination, and prolonged execution scenarios.

9. GitHub Repository

GitHub Repository Link: https://github.com/poojithamusunuri/Thread-Runners_OSSP/tree/main/ForgeOS

Repository Created: _____

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: _____

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____